

Assignment: Unified Software Requirements Specification (SRS) & Project Plan

Part I: Front Matter & Governance

1. Document Header & Versioning

Formal Title: Visually Represented Argument Parsing and Analysis for Media Literacy

Team Number: 22

Project Title: LitLens

Version: 0.1 Baseline

2. Personnel & Roles

Name	McMaster Email	Primary Technical Disciplines
Christina Sauro	sauroc@mcmaster.ca	Backend Engineer, Researcher
Shelby Liu	liu1690@mcmaster.ca	ML Infrastructure, Researcher
Lia Dirisio	dirisiol@mcmaster.ca	Backend Engineer, Researcher
Scarlett Cleary	clears2@mcmaster.ca	ML Infrastructure, Project Manager
Manav Rastogi	rastom1@mcmaster.ca	Frontend/Backend API Integration, Extension Infrastructure
Aditya Sahni	sahnia3@mcmaster.ca	Frontend/Backend API Integration, Extension Infrastructure

3. Definitions, Acronyms & Abbreviations

Argumentation Mining: Automatic identification of an argumentative structure in text via argumentative components and relationships between them.

Attack relation: A relation in which one component (typically premise) argues against another component (typically claim). Once again this is describing the argumentative structure only.

Claim: Statement in an article that represents an opinion or conclusion the author wants the reader to accept.

ML: Machine Learning.

NLP: Natural Language Processing.

Premise: A reason or piece of evidence for or against another component (Note: we currently treat evidence as a type of premise with separation as a possible extension).

Support relation: A relation in which one component (typically premise) gives reason to accept another component. Relations describe structure only, not factual accuracy.

Toulmin's Model of Argumentation: Logical framework that is used to define the different components of an argument. There are six elements: Claim, Grounds/Data, Warrant, Backing, Qualifier, and Rebuttal. Our project employs a simplified model (Claim, Premise(Grounds/Data) and Attack relations(Rebuttal)).

4. Contributions Table

Team Member	Sections Authored	Sections Reviewed	Estimated Percentage Contributed
Lia Dirisio	10, 11, 17	1, 2, 8	18%
Christina Sauro	5, 6, 13	8, 3, 10, 12	18%
Shelby Liu	3, 7, 9, 15	8, 5, 11, 16	18%
Scarlett Cleary	7, 9, 10, 14, 16	1, 3, 8, 15	18%
Manav Rastogi	7, 9, 15	8	14%
Aditya Sahni	9, 10, 14		14%

5. Executive Summary & Purpose:

Through political news media, it can be difficult for users to recognize when an argument is properly supported and verified. This can potentially lead to information disorder through channels such as social media and by word-of-mouth. By defining a clear logical argument structure for an article, content can be more comprehensible as it is broken into concise components including claims and premises.

The purpose of LitLens is to create an educational tool for users to be able to break down political media into valid argumentation in a format that is easy to understand. Through a web browser extension, the user can select a news article and identify claims in the article and whether they are properly supported based on a simplified version of Toulmin's Model of Argumentation. Claims will be gathered from the article, and the article will be checked for sufficient premises for each claim. The software will not only indicate where the overall claim and supported evidence is located in the article, but will also visualize to the user how claims are supported or unsupported by different sources using a chart structure. This in turn helps users visualize the logic structure of arguments given in a news article. Further, if a claim is not sufficiently supported, they are able to understand the rationale behind the argument not being properly proven. There will additionally be planned frontend features that encourage the user to perform extended research outside of the project. For example, if evidence is found in the article, the software will ideally provide suggested queries for the user to research and verify on their web browser.

As an easily accessible tool through the browser, LitLens will be valuable to users aiming to better understand and analyze political media in any form: for research, for journalism, or even general web browsing. By being able to break down claims and understand the logical reasoning used behind them, the project would aim to promote media literacy for individuals consuming political media on a regular basis. After implementation, the project would ideally encourage users to be able to actively conceptualize the logical reasoning of a news article when reading them in their day-to-day lives.

Part II: Software Requirements Specification (SRS)

6. Stakeholders & User Personas

This tool is meant to encourage media literacy and research for users who browse political content on a regular basis. Primary clients would include researchers and academics who seek political news articles for their work and people who read political news articles in their personal time. These clients would be looking to gain additional insights by being able to deconstruct the larger ideas of articles into a concrete logical reasoning structure.

Secondary stakeholders, who would have a more indirect influence on the project, would include news outlets that produce political media. This group would also include the project supervisor, Dr. Allison Lahnala, as her input may influence the project functionality, features, and extensions.

End-user archetypes would include individuals who engage with political media and are motivated to understand political news articles fully from the article author. The users would have a digital background and are familiar with browsing news articles through websites. End-users would have the goal of retaining valuable insights through political media and having a grasp of events occurring in their community.

Dr. Lahnala is an external supervisor and has an active role in the project's development. Her expectation of the project is that it has an overall impact of avoiding information disorder and supporting media literacy for the general population.

7. Constraints & Assumptions

Target Platform & Operating Systems:

The backend and ML pipeline run in a Linux (RHEL 10) container, to ensure all teammates have a consistent environment. The extension targets desktop Google Chrome 114+ on Windows, macOS, Linux and ChromeOS. Mobile browsers, Safari and Firefox are not supported.

Hardware Requirements: Users need a machine that runs Chrome (around 4 GB RAM) and an internet connection. Training requires 12-16 GB to fine-tune BERT models which is satisfied by the department's NVIDIA GPU servers.

Cost Constraints: The baseline is \$0, using free department GPUs and the CAS server. Publishing to the Chrome Web Store has a one-time \$5 USD developer fee.

Architectural Boundaries:

- The extension only extracts the article, calls the API, and displays the results. Communication is only via the versioned JSON schema over HTTPS.
- Manifest V3 limits: No remote code, and the service worker shuts down after ~30s idle. Permissions are minimal, chrome:// pages, the Web Store, PDFs, and paywalled or iframe content cannot be read.
- Permissions: Only activeTab, scripting, sidePanel, storage and our API domain. Pages are read only when the user clicks analyze.
- Input limits: Articles are capped at 5,000 words, and the user is warned if text is cut off.

Assumptions: Users are on an up-to-date desktop Chrome with internet access, articles are in English, news sites use reasonably standard HTML (e.g. <article> tags or NewsArticle metadata), and the backend API is available over HTTPS.

IP considerations: Our training data is used under each dataset's license column (see section 9). Our product is purely non-commercial and academic which follows all the licensing restrictions of our selected datasets. For datasets that prohibit redistribution we will not commit them to our repository and will instead provide download instructions and its applied preprocessing scripts. Additionally, the article contents analyzed by our tool will never be redistributed.

8. Functional Requirements (P0 to P3 Prioritization)

Description	Input	Output	Data flow	Focus
<i>P0 (Must Have)</i>				
The web extension extracts the main article text from the current webpage.	Current webpage (HTML)	Clean article text with its headline, sent to the backend API	Content script -> Background service worker -> backend API	Frontend
Main claim of article is identified using article headline/text	Headline and article text	The main claim of the article	Raw text->Text Preprocessing -> Model -> Backend response	Backend
Our system classifies the argumentative components of the article (Claim, Premises, Non-argumentative).	Cleaned article text	The argumentative components (Claims and Premises)	Cleaned text-> -> Component classifier-> Backend response	Backend
Our system extracts the relationship between Claim and premise (Support, Attack, Neither).	Argumentation components (Claim, Premise)	The relationship between the components (X premise is attack/supporting Y claim)	Labelled components -> Relation classifier -> Backend response	Backend
The web extension displays the model's output of main claim and premises with relations in the side panel.	Backend response with parsed argument structure	Side panel listing the claims and premises with their relations, plus loading and error states	JSON response -> React side panel UI	Frontend
The argumentative structure is visualized as a type	Backend response with parsed	Graph visualization representing the parsed argumentation structure	JSON response -> Graph data conversion ->	Frontend

of hierarchical graph via our web extension.	argument structure		Graph visualization in side panel	
The extension shall remind users that the map shows argument structure only and encourage them to verify claims with outside sources.	Backend response	Message in side panel informing user and encouraging external verification	Backend response -> side panel message displayed	Frontend
P1 (High Priority)				
The web extension uses text highlighting and UX-friendly indicators of evidence in claims (e.g., supporting evidence highlighted in green).	Character offsets of claims and premises from the API	Highlights in the article, using colour plus an icon or underline so they don't rely on colour alone	Character offsets in JSON -> content script -> Highlights on the page	Frontend
The web extension uses query generation to suggest further research to users on claim.	Backend response with parsed argument structure (Argumentation components and relations)	A list of string queries that the user can search in their web browser	Main claim -> query generation module -> JSON response -> side panel update	Backend, Frontend
We extend our system to identify additional components of Toulmin's Model like Backing to our argumentation scheme	Cleaned article text	The argumentative components (Claims, Premises, Backing)	Cleaned text -> Text preprocessing -> Component classifier -> Backend response	Backend
P2 (Medium Priority)				
We identify embedded claims in the article (rather	Article and Argumentation	Support/Attack relationship not just premise to claim but	Cleaned text -> Text preprocessing -> Component	Backend

than just the main claim) and extracts relationships between claims.	components	also claim -> claim relationships	classifier-> Backend response	
Our system employs claim verification via retrieved external evidence	Main claim of the article	If the claim is true, false, uncertain etc	Main claim-> Evidence retrieval -> Verification of results -> Backend response -> sidepanel	Backend/ Frontend
The model identifies information disorder (malinformation, disinformation, misinformation) in articles by identifying intent of text.	Article and argumentation components	Support/Attack relationship and intent of claims in article	Data involving claims and evidence -> Text pre-processing -> Model performance -> Performance evaluation	Backend
P3 (Low Priority)				
Applying our product in a research perspective	Argumentation components, relationships between components, evaluation results	Open-source research questions as applications from the project output	Data involving model performance/evaluation results -> research analysis/findings	Research

9. Data Strategy & Quantitative Metrics

Data Strategy Overview: For our project we require annotated data to fine-tune a pretrained model for Argumentation mining in supporting media literacy. Our data falls into three parts. 1) *Argument Component Identification* which requires data annotated with claims and premises(reasoning/evidence), 2) *Argumentative relationship Identification* which requires data labeling relations between our components as Support, Attack, or None. 3) *claim verification/information disorder classification data* for possible extensions to our project. For claim verification we may use RAG techniques by generating search questions to retrieve supporting/contradictory evidence from external sources. We may evaluate the success of RAG with fake news datasets included in the table below. Due to limited amounts of fully annotated political news articles datasets we plan to employ transfer learning strategies from related domains (student essays, political debates, editorials etc). Finally if needed we may hand-annotate a small set of news articles (around 25-30) as a “gold-standard” test set covering our planned schema to use for final evaluation only.

Data preprocessing and splits: Our data preprocessing will entail combining our corpora into a unified dataset with component labels (*Claim, Premise, Non-Argumentative Component*), and relation labels (*Support, Attack, None*). This preprocessing will include mapping each corpus's original labels to our schema. Our data will be split 80/10/10 into training, validation, and test sets. Our training set will be used to fit our model, validation set used for hyperparameter tuning/model comparisons and the test set will be held out untouched and only used once to report our final evaluation scores. Additionally, for small datasets we may employ k-fold cross-validation techniques on the training and validation data. We may also use specific datasets to perform intermediate fine-tuning for specifics like Argumentation Component Classification.

Dataset	Source (research paper, etc.)	Dataset Domain/Size	Additional notes (schema etc)
<i>Datasets below for training, validation and testing our model for identifying both argumentative components and relationships</i>			
AAEC	Stab & Gurevych (2017)	<u>Domain:</u> Student essays <u>Size:</u> 402 essays	<u>Components:</u> Major Claim, Claim, Premise <u>Relations:</u> Supports, attacks, none
LIARArg	Wang et al. (2025)	<u>Domain:</u> Political news claims <u>Size:</u> 2,832 statements	<u>Components:</u> Claim, Premise <u>Relations:</u> Support, Attack
ElecDeb 60to20	Goffredo et al. (2023)	<u>Domain:</u> Political debates <u>Size:</u> 44 debates	<u>Components:</u> Claim, Premise <u>Relations:</u> Support, Attack
<i>Datasets below for argumentation component identification. Can be used as initial pre-training of our model to better identify argumentative components.</i>			
Webis-E editorials-16	Al-Khatib et al. (2016)	<u>Domain:</u> news editorials from sources like Al Jazeera, Fox, The Guardian etc <u>Size:</u> 300 articles	<u>Components:</u> common ground, assumption, testimony, statistics, anecdote
WebDiscourse Corpus	Habernal & Gurevych (2017)	<u>Domain:</u> Web discourse documents (forums, comments, posts etc) <u>Size:</u> 340 documents	<u>Components:</u> Claim, Premise, Backing, Rebuttal, Refutation
PERSU ADE 2.0	Crossley et al. (2023)	<u>Domain:</u> Student Essays <u>Size:</u> 25,000	<u>Components:</u> Lead, Position, Claim, Counterclaim, Rebuttal, Evidence, Concluding Summary, Unannotated.
argumet explicati on	Ankita Gupta et al. (2024)	<u>Domain:</u> Internet Comments (Political - related to Covid-19 Vaccine approval for children)	<u>Components:</u> claim, reason, warrant (including 2 candidate warrants and an indication of which one is correct)

		<u>Size</u> : 226 (human annotated)	
ArgumentReasoningComprehension	Habernal et al. (2018)	<u>Domain</u> : NYT debate conversations <u>Size</u> : 1,970 arguments	<u>Components</u> : Claim, Premise <u>Relations</u> : W(correct implicit warrant),AW(alternative warrant)
<i>Dataset below for fake news clarification</i>			
Kaggle Politifact Fact Check Dataset	N. Sakib, et al (2025)	<u>Domain</u> : Political media claims <u>Size</u> : 21,152	<u>Components</u> : Claim, Verdict

Dataset Licensing: All datasets require citation or attribution and are used for non-commercial academic research only. AAEC, WebDiscourse, and LIARArg (no specified license) cannot be redistributed. ElecDeb60to20 (CC BY-NC-ND 4.0), PERSUADE 2.0 (CC BY-NC-SA 4.0), argument_explication (GPL v3), and ArgumentReasoningComprehension (Apache 2.0) place conditions on sharing modified versions. Webis-Editorials-16 and Kaggle PolitiFact (CC BY 4.0) have no further restrictions.

Quantitative/Evaluation Metrics: Model metrics are reported on the held-out test split and compared against a majority-class baseline. Macro-F1 is the primary metric because our labels are imbalanced (far more Support than Attack relations). Targets are set below published results on student-essay corpora (e.g., Stab & Gurevych, 2017) because political news text is longer, less structured and harder to annotate.

Metric	Target
Argument component identification (Claim / Premise / None), macro-F1	≥ 0.70
Relation classification (Support / Attack / None), macro-F1	≥ 0.60
Main claim identification accuracy (50 hand-labelled news articles)	$\geq 70\%$
End-to-end latency for a ~1,000-word article (click to graph shown)	≤ 10 s
Article text correctly extracted from news pages	$\geq 90\%$
API error rate	$< 2\%$
User satisfaction, System Usability Scale (SUS)	≥ 68
Accessibility (WCAG 2.1 AA)	0 critical issues

10. Non-Functional Requirements (NFRs)

Usability & Human-Computer Interaction

The browser extension shall provide an intuitive interface for viewing the argument analysis and understanding the resulting chart. The visualization implemented shall use consistent labels, symbols, and relationships to help users understand the Argumentation components present. In addition, the system shall provide contextual help in the form of tooltips, chart symbols, and relationships to help users understand the visualization. To aid user understanding of the operations of the system, clear feedback will be provided when an article is being analyzed, or when error occurs. The browser extension shall prevent user error by not allowing a user to run the article analysis on a webpage that does not include an article. The system shall minimize cognitive load by presenting a simple hierarchical chart. The system shall provide accessible text labels, visual contrast, and non-colour-dependent indicators where applicable, following relevant WCAG guidelines. The system shall provide clear error messages when an article cannot be analyzed or the argument structure and components cannot be identified.

Performance, Scalability & Speed

The system shall analyze an article of typical length and generate the argument visualization within a reasonable processing time (maximum 10-15 seconds). For longer articles and articles with multiple arguments, the system's process time should not reflect significant degradation in performance. Further, the system shall process only the intended articles to avoid unnecessary processing time. User zooming, panning, selecting, and navigating the chart should remain responsive as chart complexity increases. The system will define a maximum article length (5,000 words) to prevent extremely large inputs from causing excessive processing times or system failures.

Security, Authentication & Data Privacy

The system shall protect article content from unauthorized access. When transmitting data between the browser extension and backend processing, the system shall use secure communication protocols. The system shall not collect or store any user data as part of normal operation. The system shall only process the article content necessary to perform argument analysis and shall not collect unrelated webpage or browsing data. Before processing, the system shall validate and sanitize article content and other necessary inputs to reduce security risks. If user feedback is collected, the system shall protect the feedback from unauthorized access and clearly inform users about its collection and intended use.

Reliability, Fault Tolerance & Maintainability:

Reliability: Given identical article text and the same model version, the backend shall return the same components, relations, and character offsets. The backend API will be available with a target of 99% availability and an API error rate below 2% during demos and user testing (Section 9), and a health-check (GET /health) will report if the service is up and the model is loaded, which the extension will check before submitting an analysis.

Fault Tolerance: The system shall fail gracefully and visibly. Requests time out after 30s, after with an error message and a retry option in the side panel. An extension failure shall never alter

or break the host page, Shadow DOM highlights are removable in one action. In-flight request state is kept in `chrome.storage.session` so work survives MV3 service worker restarts. The backend rejects oversized, malformed or non-English input with a descriptive 4xx, and enforces a per-request time limit. If a later stage of the pipeline fails, the service returns the partial result with a warning flag, which the extension labels as incomplete. The container auto-restarts and logs never contain article content.

Maintainability: The extension and the backend are independently deployable, joined only by a versioned JSON API. The backend stays backward-compatible with the previous minor version for one release. Code quality is enforced with ESLint and Prettier, type hints with Ruff, and the test suites are executed automatically on every pull request through GitHub Actions with at least 70% line coverage on core modules. Environment-specific values are read from configuration; dependencies are pinned via Docker; and each model release ships with a model card (data, metrics, limitations).

11. Risk Management & Feasibility Analysis:

Risk	Likelihood	Severity	Potential Impact	Mitigation Steps
ML model struggles to accurately identify argument components	High	High	Incorrect components reduce usefulness	Start with a baseline model, fine-tune the model to relevant data. Additionally, we can employ pre-training methods.
ML model misidentifies relations between components	High	High	Incorrectly identified relationships can misrepresent the argument structure	Pre-existing literature indicates attack relations are more difficult to identify, thus we will make sure to address class imbalance and combine our data to ensure we have enough relationship extraction data.
Chart produced by model is complex and difficult to interpret	High	High	The chart output of the application is unreadable and not useful to the user	Limit chart complexity, and prioritize the main claim and related supporting claims. Provide labels, tooltips etc
Model performance is too slow for real time use	Medium	Medium	Users may experience long wait times when analyzing articles, resulting in a poor user experience	Use a lightweight or optimized pretrained model and limit processing to the relevant article text. If processing remains slow, perform analysis asynchronously and display a loading indicator while the chart is generated.
Datasets do not reflect real	Medium	High	The model performs poorly on real news	Additionally test on a separate set of real media articles (manually

world political articles				annotated)
Limited time for ML training and system integration	Medium	Medium	We may be unable to fully implement and test the ML pipeline/browser extension, resulting in a incomplete prototype	Use pre-trained models, establish basic functionality required from the model

Part III: Project Development & Execution Plan

12. Team Communication & Meeting Cadence

Internal meetings within the project team will happen 3 times a week at the minimum during the course lecture times. These meetings will help outline goals for the week, delegate tasks to team members, provide an opportunity to check-in with tasks and address concerns, and brainstorm ideas or solutions related to the project. There may be delegated breakout group work based on the goals of these meetings. All meeting notes will be initially created on Google Docs to promote collaborative work during the meetings, then later downloaded and pushed to GitHub under the “/docs/meetings” directory.

Meetings with the project supervisor will occur frequently at the beginning stages of the project, up to twice a week. This will help the team receive guidance and fully understand requirements. As the project progresses, meetings will become less frequent and more on a case-by-case basis. Mandatory check-ins with the teaching staff of the capstone course will also be attended by the team. These check-ins will happen at least twice per semester.

13. Engineering Workflow & Version Control

Planned Repository Structure

Directory	Usage/Purpose
/src	This folder will contain core development code for the frontend and backend.
/data	This folder will contain a README outlining our data label schema, dataset splits, mappings from the original source corpora, and download instructions. We may also include a folder for data from modified datasets adhering to licensing restrictions, as well as any self-created data.
/tests	This folder is specifically for unit and integration tests, backend testing for our model, as well as frontend testing.
/docs	This folder will contain deliverables for the COMPSCI SCI 4ZP6A / 4ZP6B course.

/docs /meetings	This folder will contain information related to group meetings, meeting notes (this includes meetings within the group and meetings with supervisor) as well as meeting agendas.
--------------------	--

Branching Strategy

Issue tracking will be handled through GitHub Projects. This allows the team to easily define, group, and assign tasks accordingly. There will be a separate main branch for backend and frontend work. There will also be a branch for testing purposes. The team will ensure that any work done on branches will be extensively tested and reviewed before being merged with the main branch. Further, any pull requests on GitHub must be properly documented with changes and the intended impact on main code.

Scrum Framework Adoption

The team's scrum framework will include weekly sprints with a weekly meeting to discuss progress including deliverables, challenges, and obstacles. There will be short 50-minute weekly meetings during the lecture times of the capstone course (Monday, Wednesday, and Thursday at 1:30pm-2:20pm). There will also be monthly check-ins for more larger-scale deliverables. This ensures that the team is on track with deliverables regarding frontend and backend progress. The team will also have weekly meetings with the project supervisor to provide updates on progress for milestones and to ask clarifying questions. These supervisor meetings will happen once a month at minimum. Backlog tasks for the project will be managed through the GitHub Projects tab. These tasks will be grouped together and managed based on priority. During team meetings, applicable backlog tasks will be discussed and assigned to a team member based on impact of task, time to complete task, and how critical it is to the project's functional requirements.

14. Infrastructure & Compute Resources:

Training and Fine-tuning

Models will be trained and fine-tuned utilizing the department's GPUs (grace, ada, and s4090a). To connect to the GPUs we will SSH into the server using: "ssh [hostname].cas.mcmaster.ca". While training, if we find the GPUs tend to be at 100% capacity, we will look into getting a Digital Research Alliance of Canada account sponsored by our supervisor as a backup.

Web Extension Infrastructure

During development the extension is built with Vite and loaded unpacked through chrome://extensions in Developer mode. Test builds are produced by GitHub Actions on each tagged version and attached to GitHub Releases as a zip that the supervisor and study participants can sideload. The final version will be published to the Chrome Web Store under a shared team developer account. Store review requires a Manifest V3 package, a justification for each permission (activeTab, scripting, sidePanel, storage, and the API host), and a privacy policy URL, which will be hosted on GitHub Pages and state that article text is processed in memory and never stored (Section 10). A Vite environment variable selects the production API domain for release builds and localhost for development builds.

Backend API Hosting

The FastAPI inference service (Section 15) runs as a Docker container on a CAS departmental virtual server, requested through the department with Dr. Lahnala's support. Uvicorn runs behind an nginx or Caddy reverse proxy with a Let's Encrypt certificate. HTTPS is required because Manifest V3 extensions can only call secure origins and the host permission is tied to one API domain, so the team will also request a stable DNS name for the server. Inference runs on CPU: a BERT base model with adapters handles a 1,000 word article within the 10 second target from Section 9 on 2 to 4 vCPUs and 8 GB of RAM, so no GPU is needed for serving. Model weights are included in the image at build time, and deployment is a pull and restart on the VM with a restart policy for crash recovery (Section 10). If a CAS server cannot be provisioned before the PoC demonstration, the fallback is a small cloud VM (2 vCPU, 8 GB, roughly US\$30 to 60 per month) running the same image.

15. Technology Stack & Architectural Rationale

The system has two parts joined by a small HTTPS JSON API: a Chrome extension (frontend) that extracts the article and shows results, and a Python inference service (backend) that runs the fine tuned model. Keeping the model on a server keeps the extension light and lets the model be updated without republishing the extension.

Languages

Language	Role
TypeScript	All extension code (content script, service worker, side panel UI)
Python	Backend API and ML pipeline
JSON	Contract between extension and API

Frameworks (Frontend and Backend)

Framework / Library	Role	Rationale
Chrome Extension Manifest V3	Extension platform	Required for new Web Store extensions; side panel API keeps the chart beside the article
React, Vite	Side panel and popup UI, build tool	Component model suits repeated elements (claim cards, legends); Vite bundles for MV3 with fast reloads
Cytoscape.js/D3.js	Argument chart	Javascript libraries build for interactive graphs: layouts, zoom, pan, edge styling by relation type
Tailwind CSS	Styling	Consistent spacing and WCAG contrast; Shadow DOM highlights avoid clashing with site styles
FastAPI, Pydantic, Uvicorn, Docker	Backend API	High performance Python API; Pydantic enforces the shared JSON schema; Uvicorn runs it on the CAS server. Same environment on every machine and on the server.

ML Toolkits

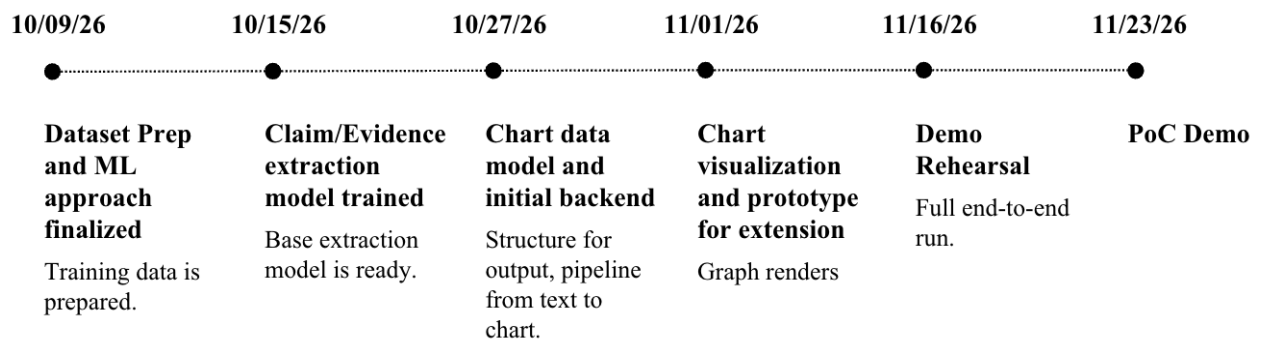
Toolkit	Role
Hugging Face Transformers, AdapterHub	Pretrained BERT models, tokenizers, classification heads; adapters for transfer learning
PyTorch	Training and running models
Scikit-learn, spaCy, pandas	Preprocessing: sentence segmentation, merging and mapping corpora. Evaluation: F1, recall, confusion matrix.

Unit and Integration Testing Suites

Tool	What it tests	Rationale
Pytest, FastAPI TestClient	Backend units and integration: preprocessing, label mapping, splits, API handlers, output schema, request to model to JSON response	Python standard; fixtures and coverage plugins. Tests the real endpoint in process without deploying
Vitest	Frontend units: response parsing, offset to highlight mapping, chart data	Native to Vite, Jest compatible
React Testing Library	Components: claim cards, legend, loading and error states	Tests by text and role, which also encourages accessible markup
Playwright	End to end: loads the extension on saved news pages, checks extraction, API call, chart and highlights	Supports Chrome extensions; measures the 90% extraction and 10 s latency targets from Section 9

16. Proof of Concept (PoC) Demonstration Plan

Roadmap to Demonstration



Demonstration Scope

Technical Risks	Working Code	Interactive Features
Component extraction: missing argument components	Fine-tuned extraction model	Input text and see argument components highlighted
Relationship identification: class imbalance	Relationship extraction with balanced training data	The extraction feature also outputs relationships between components.
Element selection in browser	Browser extension prototype	Click on an article and the extension will identify the target text
Chart interpretability: user misunderstanding of components	Chart visualization	Visual shown to users of the argument elements and relationships.
Model errors appear in chart	Backend and graph data model	Flag for low-confidence
Real-time performance	Optimization	Live end-to-end run from the extension, showing response time

17. Project Schedule & Gantt Chart

The Gantt chart below presents the planned development schedule for the project. The x-axis is organized by week, with each date representing the start of that week. The project timeline is divided into phases, with each phase corresponding to a deliverable. This organization provides a clear overview of the progression of the project and the expected completion of tasks.

